

Code Explanation

Test DLT Pipeline

This test suite is designed to validate the functionality of the DLT pipeline transformations. It includes tests for the `order\_silver` and `customer\_silver` transformations, ensuring that they produce the expected output.

The test class `TestDLTPipeline` contains two main test methods: `test\_order\_silver\_transformation` and `test\_customer\_silver\_transformation`. These tests cover the transformation logic for order and customer data, respectively.

Overview of the Code

The provided code is a test suite written in Python using the unittest framework. It tests the functionality of two data transformations: `order\_silver` and `customer\_silver`, which are part of a Data Lake Transformation (DLT) pipeline. The tests are designed to validate the correctness of these transformations by comparing their output with expected results.

Step 1: Setting up the test environment

The test class `TestDLTPipeline` sets up a Spark session and creates sample customer and order data. This step is crucial for creating a test environment that mimics the actual data processing pipeline.

```
@classmethod
def setUpClass(cls):
    cls.spark = SparkSession.builder
        .appName("TestDLTPipeline")
        .master("local[*]")
        .getOrCreate()
```

Step 2: Creating sample data for testing

The test class creates sample customer and order data using Spark DataFrames. The data includes various scenarios, such as different regions and order details.

```
customer_data = [
    ("C001", "John Doe", "john@example.com", "North"),
    ("C002", "Jane Smith", "jane@example.com", "South"),
    ("C003", "Bob Johnson", "bob@example.com", "East"),
    ("C004", "Alice Brown", "alice@example.com", "West"),
]

customer_schema = StructType([
    StructField("CustId", StringType(), True),
    StructField("Name", StringType(), True),
    StructField("EmailId", StringType(), True),
    StructField("Region", StringType(), True)
])

cls.customer_df = cls.spark.createDataFrame(customer_data, schema=customer_schema)
```

Step 3: Testing the `order\_silver` transformation

The `test\_order\_silver\_transformation` method tests the `order\_silver` function by mocking the `dlt.read` function to return the test order data. It then calls the `order\_silver` function and verifies that the output is correct.

```
@patch('dlt.read')
def test_order_silver_transformation(self, mock_dlt_read):
    from src.dlt_pipeline import order_silver
    mock_dlt_read.return_value = self.order_df
    result = order_silver()
    laptop_row = result.filter(result.ItemName == "Laptop").collect(
    )[0]
    self.assertEqual(laptop_row.TotalAmount, 2000.0)
```

Step 4: Testing the `customer\_silver` transformation

The `test\_customer\_silver\_transformation` method tests the `customer\_silver` function by creating test data with nulls and duplicates. It then mocks the `dlt.read` function to return this test data and calls the `customer\_silver` function. The test verifies that nulls are removed and duplicates are handled correctly.

```
@patch('dlt.read')
def test_customer_silver_transformation(self, mock_dlt_read):
    from src.dlt_pipeline import customer_silver
    test_data = [
        ("C001", "John Doe", "john@example.com", "North"),
        ("C002", "Jane Smith", "jane@example.com", "South"),
        ("C003", None, "bob@example.com", "East"),
        ("C001", "John Doe", "john@example.com", "North")
    ]
    schema = StructType([
        StructField("CustId", StringType(), True),
        StructField("Name", StringType(), True),
        StructField("EmailId", StringType(), True),
        StructField("Region", StringType(), True)
    ])
    test_df = self.spark.createDataFrame(test_data, schema=schema)
    mock_dlt_read.return_value = test_df
    result = customer_silver()
    self.assertEqual(result.filter(result.Name.isNull()).count(), 0)
```

Step 5: Tearing down the test environment

The `tearDownClass` method stops the Spark session after the tests are completed.

```
@classmethod
def tearDownClass(cls):
    cls.spark.stop()
```